
CS 170 Midterm 1

Write in the following boxes clearly and then double check.

Name :

SID :

Exam Room :

- ☐ Pimentel 1
- ☐ Wheeler 20
- ☐ Other (Specify):

Name of student to your left :

Name of student to your right :

- The exam will last 110 minutes.
- The exam has 4 questions, each worth 25 points, plus a challenging bonus question worth only 1 point.
- Answer each question in the box provided. If you need more space, continue on the reverse side of the page and clearly state that you are doing so in the box.
- Good luck!

Use this page for scratch work.

1 [25 points] Mining

Deep in a mine, a miner is trying to choose which minerals to carry back out. There are n different minerals, each with some quantity q_i (measured in pounds) available, and some value v_i per pound. The miner wants to bring back minerals with the maximum possible total value, but can only carry 100 pounds back up. In this problem, you will design an efficient algorithm that computes the maximum value of minerals they can carry subject to this constraint.

For each mineral i , the miner may take any fraction of the available q_i pounds, subject to the overall 100 pound maximum. That is, there is no restriction to integer values, and you may analyze running time assuming each real number arithmetic operation takes one timestep.

- (a) Give an algorithm to maximize the miner's value, and analyze the runtime.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

- (b) Prove correctness using an exchange argument.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Solution:

- (a) We sort the n minerals by decreasing order of value per pound, v_i . We then greedily take all of the mineral with highest value per pound, then with the second highest value per pound, then the third highest, etc. until we reach 100 total pounds, or else run out of available minerals with positive value. The running time is $O(n \log n)$ to run sorting.
- (b) For the exchange argument, assume for contradiction that greedy is not optimal. Consider all optimal solutions sorted by value per pound — now choose an optimal solution that is as close to the greedy solution as possible in the following sense: it is identical to the greedy solution for the first j pounds for j as large as possible. Now this optimal solution and greedy differ on the $j + 1$ pound. But since greedy chose the best value per pound available (and since the same minerals were available to optimal and greedy at this point), if we replace the $j + 1$ -st pound of optimal solution with the $j + 1$ -st pound of greedy, we would increase the value of the optimal solution, thus contradicting optimality.

Use this page for scratch work.

Subproblem: Let $f(i)$ be the maximum possible total reward when restricting to jobs starting at time $\leq s_i$.

Recurrence: If we do not use job i , then the maximum reward for our subproblem is $f(i-1)$. If we do use job i , then we can only use other jobs starting before time $s_i - 1$. If we define j_i to be the last job starting before time $s_i - 1$, then in this case the maximum reward is $f(j_i) + r_i$. Therefore

$$f(i) = \max\{f(i-1), f(j_i) + r_i\}.$$

Base case: $f(0) = 0$, where we define $s_0 = -\infty$.

Output: $f(n)$.

Runtime: Computing $f(i)$ requires two table lookups (for $O(n)$ total steps) + computing j_i , which requires looking up $j_i - j_{i-1}$ values of s_k (for total of $\sum_i j_i - j_{i-1} = j_n \leq n$ steps).

Use this page for scratch work.

3 [25 points] Shortest Path

You are given a weighted directed graph $G = (V, E)$ in which one edge has weight -1 , and all other edges have weight $+1$. You want to find the shortest path from a given source vertex s to a target vertex t .

- (a) Give a $O(|V| + |E|)$ time algorithm for this problem, which takes as input the graph $G = (V, E)$, vertices s, t , and the unique edge (u, v) with weight -1 . Your algorithm should use breadth-first search.

.....

.....

.....

.....

.....

.....

.....

.....

.....

- (b) Justify the correctness of your algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

Solution:

- (a) Remove edge (u, v) . Then run BFS twice on the resulting graph G' , once from source s and once from source v , while storing the length of the path from the sources. Output the minimum of:

- (1) The distance in G' from the s to t , and
- (2) The distance in G' from s to u plus the distance from v to t minus 1.

Each BFS, and hence the overall algorithm, uses time $O(|V| + |E|)$.

- (b) **Case 1:** The shortest path in G from s to t does not use edge (u, v) : in this case it is the shortest path in G' from s to t , which we compute in case (1) above.

Case 2: The shortest path in G from s to t does use edge (u, v) : in this case it must first traverse a shortest path in G' from s to u , then traverse (u, v) , and then traverse a shortest path in G' from v to t , which we compute in case (2) above.

Use this page for scratch work.

4 [25 points] Decimal to Binary

Given the n -digit decimal representation of a positive integer, converting it into binary in the natural way takes $O(n^2)$ steps. In this problem, you will design a more efficient divide-and-conquer algorithm for converting decimal to binary.

Let n be even. Then any n -digit integer can be written as $a \cdot 10^{n/2} + b$ where a and b are both $n/2$ -digit integers, e.g. $1342 = 13 \cdot 10^2 + 42$.

- (a) Give a divide-and-conquer algorithm to convert an n -digit number to binary.

.....

.....

.....

.....

.....

.....

.....

.....

- (b) Argue that the running time of your algorithm in part (a) is $O(n^{\log_2(3)} \cdot \log(n))$.

It may be helpful for you to recall that the Karatsuba algorithm for integer multiplication takes $O(n^{\log_2(3)})$ steps to multiply two n -bit integers given in their binary representations.

And, as in Karatsuba, you may assume for simplicity for this question that n is a power of 2.

Note: there is a trick to improving the running time to $O(n^{\log_2(3)})$, so your runtime could be faster than what the question asks for.

.....

.....

.....

.....

.....

.....

.....

.....

Solution:

- (a) Writing $x = a \cdot 10^{n/2} + b$ where a and b are $n/2$ -digit integers, we recursively apply our algorithm to a , $10^{n/2}$, and b thereby obtaining binary representations of a , $10^{n/2}$, and b . We then multiply these binary representations of a and $10^{n/2}$, and add in the binary representation of b .
- (b) The binary multiplication takes $O(n^{\log_2(3)})$ time, and the addition takes $O(n)$ time. Setting up the subproblems also takes $O(n)$ time, as a and b simply consist of the first and last $n/2$ digits in x , respectively. Therefore the running time satisfies $T(n) = 3T(n/2) + O(n^{\log_2(3)})$ time, so by the Master Theorem $T(n) = O(n^{\log_2(3)} \log n)$.

You could alternately recurse only on a and b , and compute the binary representation of $10^{n/2}$ separately in $O(n^{\log_2(3)})$ time as follows: repeatedly square (in binary) 10 , $10^2 = 10 \cdot 10$, $10^4 = 10^2 \cdot 10^2$, $\dots$, $10^{n/2} = 10^{n/4} \cdot 10^{n/4}$. The point is that these numbers double in length with each squaring, so the running times form a geometric series which sums in big-Oh to just the last term, which is $O(n^{\log_2(3)})$ by Karatsuba. Then our divide-and-conquer algorithm only has two subproblems (for a, b), giving recurrence $T(n) = 2T(n/2) + O(n^{\log_2(3)})$, which has solution $T(n) = O(n^{\log_2(3)})$.

Use this page for scratch work.

Solution: Key observation: Suppose we change the weight of all free edges to equal some value x . If the MST T_x of the resulting graph G_x uses exactly k free edges and has total weight W , then we observe that the minimum weight MST with at most k free edges in the original graph must have weight $W - kx$. i.e. T_x is the tree we are looking for.

Proof of key observation: Suppose for contradiction that there is some spanning tree T' for G using $\leq k$ free edges, whose non-free edges have total weight $W' < W - kx$. If we set the weight of every free edge to equal x , then T' has total weight $\leq W' + kx < W$. This contradicts the fact that T_x was an MST of G_x .

Algorithm: By the key observation, we want to find some value of $x \geq 0$ for which the MST of G_x uses exactly k edges in F . Consider continuously increasing the weight x of the free edges until their number in the MST decreases to exactly k . There are two issues we must address: 1) is it possible that the number of “free” edges in the MST ever changes by more than 1 as we increase x , possibly skipping over k entirely 2) Can we speed up this continuous algorithm so we test only $O(n)$ or possibly only $O(\log n)$ different values of x .

Assume for simplicity that all non-free edges have distinct weights w_i in sorted order (otherwise we could slightly perturb the weights to break ties). Then as we increase x from some value between w_{i-1} and w_i to some value between w_i and w_{i+1} , the i -th edge might displace a “free” edge from the MST (it follows from the proof of Kruskal that no more than one free edge can be displaced by that change in x). So it suffices to consider n such weights for x . Clearly, it is sufficient to do a binary search on these n values instead.

Running Time: We can find the index i with a binary search using $O(\log |E|) = O(\log |V|)$ iterations. Each iteration must find the MST of G_x for some x , which takes $O(|E| \log |V|)$ time using Kruskal’s algorithm, for a total time of $O(|E| \log^2 |V|)$.

We can improve this running time by pre-sorting all non-free edges in time $O(|E| \log |V|)$. Then for each value of x , we can insert the free edges of weight x into the sorted array in time $O(|E|)$, and finish Kruskal’s algorithm in time $O(|E| \log^* |V|)$, for a total running time of $O(|E| \log(|V|) \log^*(|V|))$.

Use this page for scratch work.

Assumptions

(0 points) Congratulations! You're done with the exam. Were there any ambiguities in the exam? If so, record the ambiguity and the assumptions that you made in the box below, as well as possible alternate interpretations and their associated answers.

For example, "If question X is asking about A, then my answer is Y. But, if the question is about B, then my answer is Z." If you submit a regrade request regarding any ambiguities on the exam, we will only consider the request if you have correctly noted an ambiguity in the box below.

Comments, Concerns, etc.

(0 points) Did you notice anything suspicious during the exam? Please note in the box below. Where were you sitting, and what did you notice?

(0 points) Any other comments, concerns, qualms, queries, or suggestions?

(0 points) 100 penguins versus one CS 170 student. Who would win? Justify your answer.